NXP Semiconductors
Release Notes

Document Number: S32DS360
Rev. 1.0, 11/2024

S32 Design Studio for S32 Platform 3.6.0 RFP

Contents

1. Read Me First

NXP Semiconductors is pleased to announce the release of the S32 Design Studio for S32 Platform 3.6.0 for NXP Arm® based devices and hardware accelerators. S32 Design Studio for S32 Platform is based on the Eclipse open development platform and integrates the Eclipse IDE, GNU Compiler Collection (GCC), GNU Debugger (GDB), and other software to offer designers a straightforward development tool with no code-size limitations.

2. Release Content

2.1. Release Content

- GNU Bare-Metal Targeted Tools for Arm® 32-bit Embedded Processors:
  - # GCC version 11.4 20230528, build 1763 revision gf703eb2
  - # GCC version 10.2 20200723, build 1728 revision g5963bc8
  - # GCC version 9.2.0 20190812, build 1649 revision gaf57174
- GNU Bare-Metal Targeted Tools for Arm® 64-bit Embedded Processors:
  - # GCC version 11.4 20230528, build 1763 revision gf703eb2
  - # GCC version 10.2 20200723, build 1728 revision g5963bc8
  - # GCC version 9.2.0 20190812, build 1649 revision gaf57174
- GNU Linux Targeted Tools for Arm® 64-bit Embedded Processors:
  - # GCC version 11.4 20230528, build 1763 revision gf703eb2
  - # GCC version 10.2 20200723, build 1728 revision g5963bc8
  - # GCC version 9.2.0 20190812, build 1649 revision gaf57174
- Libraries: NewLib, NewLib Nano[1]
- Semihosting for Arm® 32-bit and 64-bit bare-metal target toolchains.
- MSYS2 2022.06-1.
- Eclipse 2023-12 framework.
- GDB 15.1 with Python 3.10.x support.
- S32 Debugger (with S32 Debug Probe) support for Arm® cores.
- S32 Debugger provides OS Awareness support for FreeRTOS, OSEK and Zephyr OS.
- S32 Trace for Arm® Cortex®-A53, R52, M33, M7 cores with S32 Debugger.
- S32 Flash Tool.
- PEmicro® debugger support.
- Lauterbach Trace32® support.
- IAR debugger support.
- TASKING debugger support.
- Segger debugger support.

[1] The availability of libraries depends on the toolchain version and the core type. Tools for Arm® 64-bit processors support NewLib libraries only. Tools for Arm® 32-bit processors do not support EWL libraries for the Cortex®-R52 and Cortex®-M33 cores.

- Green Hills compiler support.
- IAR compiler support.
- Diab compiler 7.x support.
- The S32DS Extensions and Updates Tool.
- The SDK integration is provided with additional software packages. The SDK packages can be installed and updated with the S32DS Extensions and Updates Tool.
- SDK Management.
- The migration support is provided to upgrade:
    # SDK version attached to the project,
    # GCC based toolchain for project.
- S32 Configuration Tools with important additions in Pins, Clocks, Peripherals, eFUSE, IVT, DCD, QuadSPI, DDR, ICE and GTM tools.
- The wizards for creating application, library projects and projects from project examples for the supported processor families[2]
- The Getting Started page.
- Peripheral Registers view.
- Arm System Registers view.
- Watch registers view.
- Memory Spaces view.
- S32DS MMU Viewer.
- S32DS Debug Perspective.
- Threads (Zephyr RTOS) view.
- Smart card support for secure debugging.
- Multiple Elf Loading for S32 Debugger.
- Remote Probe Connection.
- Global Variables view.
- OS Details Browser view.
- Cross Trigger support:
    # UI option to configure various states(source/destination)
    # Cross Trigger state control - enable, disable, clear

3. What's New

3.1. New Features

- IDE Platform:
    # Integrated new Eclipse 2023.12, CDT 11.4 and Java17.
    # Integrated new components: S32 Trace 3.6.0, S32 Configuration Tool 1.8.
    # Integrated new Segger version **V8.10c**.
    # Updated the Dashboard UI to include additional links for extra tools in S32DS.

[2] Support for wizards and the project examples are provided in the device specific software packages.

    - # Updated the C/C++ and Debug perspectives and tailor the UI for embedded applications.
    - # Removed Import/Export of ProjectInfo.xml.
    - # Increased Heap Size to 4Gb for IDE to load all components fasters.
- Installer:
    - # New Window 11 OS official support.
    - # Install with “Administrator rights” automatically on Windows and Linux OSes to avoid scenarios when IT prevent users from installing tools.
    - # Removed GCC 6.3 from installer to reduce the size of the package.
    - # Installation for all GCC (9.2, 10.2, 11.4), PNE 5.9.2, GDB 15.1 and NPIs (S32G, S32R41, S32Z/E, S32R45, S32K1, S32K3, S32M2) by default to improve the out-of-the-box experience especially for MCU development use-cases.
- Debugger Platform:
    - # Integrated GDB with Python 3.10 in S32DS 3.6 for ARM core.
    - # New GDB 15.1.

4. Release Description

4.1. Supported Features

- Integrated updated versions of the tools packages:
    - # S32 Trace 3.6.0
    - # S32 Debugger Core 3.6.0
    - # S32 Flash Tool 2.2.5
    - # S32 Configuration Tool 1.8

5. Configuration

5.1. Recommended configuration

- PC with 2.6 GHz Intel® Pentium® compatible processor or better.
- 8 GB of RAM.
- 30 GB of disk space (when installing all product features or all updates.
- 24 GB of temporary storage (required only during the product installation).
- USB port for communications with target hardware.
- Ethernet port for communications with target hardware (optional).

5.2. Operational minimum configuration

- PC with 1.8 GHz Intel® Pentium® compatible processor.
- 4 GB of RAM.

- 25 GB of disk space.
- 20 GB of temporary storage (required only during the product installation).
- USB port for communications with target hardware.

5.3. Host operating system support

- Microsoft® Windows® 10 64-bit.

S32 Design Studio for S32 Platform 3.6.0 RFP supports all editions of the operating systems listed above and is limited only by the requirements of the Java Runtime Environment.

- Microsoft® Windows® 11 64-bit.

S32 Design Studio for S32 Platform 3.6.0 RFP supports all editions of the operating systems listed above and is limited only by the requirements of the Java Runtime Environment.

- Ubuntu LTS 20.04 64-bit

S32 Design Studio for S32 Platform 3.6.0 RFP supports all editions of the operating systems listed above and is limited only by the requirements of the Java Runtime Environment.

5.4. Recommended configuration

- PC with 2.6 GHz Intel® Pentium® compatible processor or better.
- 8 GB of RAM.
- 25 GB of disk space for installation files (full product or updates).
- 20 GB of temporary storage (required only during the product installation).
- USB port for communications with target hardware.
- Ethernet port for communications with target hardware (optional.

S32 Design Studio for S32 Platform 3.6.0 RFP Installation Guide specifies the installation prerequisites for Linux platforms.

5.5. Operational minimum configuration

- PC with 1.8 GHz Intel® Pentium® compatible processor.
- 4 GB of RAM.
- 20 GB of disk space.
- 16 GB of temporary storage (required only during the product installation).
- USB port for communications with target hardware.

6. Installation

6.1. Product Web Page

S32 Design Studio for S32 Platform 3.6.0 RFP is available for download from the product Web page at www.nxp.com.

6.2. Installation and Licensing

The S32 Design Studio for S32 Platform 3.6.0 RFP installation package contains the base tools and the installer. Support for the NXP Arm® based processor families is provided with additional software packages. The packages can be installed on the product from the S32DS Extensions and Updates tool or downloaded from the product Web page.

Note: The user needs admin/root privileges to install S32 Design Studio for S32 Platform 3.6.0 RFP

Note: The plug-ins to support third-party compilers or debuggers such as Lauterbach Trace32® are not included in the installation package and have to be installed from the corresponding sites or installation packages.

Run the installation package. The wizard will guide you through the installation process.

Note: Installation of S32 Design Studio for S32 Platform 3.6.0 RFP from the command line interface in the console or silent mode is not supported.

During installation, license activation is requested. The following activation types are supported:

- Online activation requires access to the Internet. A license activation request is sent automatically.
- Offline activation requires no Internet access. A license activation request is generated and needs to be passed to the licensing site manually. The activation response is loaded from the site to the installer manually.

New functionality can be added to S32 Design Studio for S32 Platform 3.6.0 RFP with additional software packages, updates and patches. Software packages add support for specific NXP Arm® based processor families. Updates and patches extend general functionality of the product and correct software defects.

New functionality can be added directly from the Internet or from a downloaded archive. If your computer is connected to the Internet, select **S32DS Extensions and Updates** on the **Help** menu to find and install all updates and packages available. If your computer does not have access to the Internet, you can download software packages, updates, and patches from the product page and install them from the archive files using the S32DS Extensions and Updates tool.

6.3. Starting S32DS

To start S32 Design Studio for S32 Platform 3.6.0 RFP and begin to work with it:

- In Windows, double-click the product icon on the desktop or go to the Start menu and click **NXP S32 Design Studio** > **S32 Design Studio for S32 Platform 3.6.0 RFP**. To run the product from the command line, open the terminal, navigate to the directory with the installed product, type `s32ds.bat`, and press `Enter`.
- In Linux, to run the product from the command line, open the terminal, navigate to the directory with the installed product, type `./s32ds.sh`, and press `Enter`.

6.4. Related Documentation

To learn more about the included tools, refer to the following Release Notes:

- S32 Debugger Release Notes located in `/S32DS/tools/S32Debugger/Debugger/`
- S32 Configuration Tools Release Notes located in `/Release_Notes/`
- S32 Flash Tool Release Notes located in `/S32DS/tools/S32FlashTool/doc/`
- GNU Bare-Metal Targeted Tools for Arm 32-bit Embedded Processors Release Notes located in:

```
# /S32DS/build_tools/gcc_v9.2/gcc-9.2-arm32-eabi/
# /S32DS/build_tools/gcc_v10.2/gcc-10.2-arm32-eabi/
# /S32DS/build_tools/gcc_v11.4/gcc-11.4-arm32-eabi/
# /S32DS/tools/gdb_arm/arm32-eabi/
```

- GNU Bare-Metal Targeted Tools for Arm 64-bit Embedded Processors Release Notes located in:

```
# /S32DS/build_tools/gcc_v9.2/gcc-9.2-arm64-eabi/
# /S32DS/build_tools/gcc_v10.2/gcc-10.2-arm64-eabi/
# /S32DS/build_tools/gcc_v11.4/gcc-11.4-arm64-eabi/
# /S32DS/tools/gdb_arm/arm64-eabi/
```

- GNU Linux Targeted Tools for Arm 64-bit Embedded Processors Release Notes located in:

```
# /S32DS/build_tools/gcc_v9.2/gcc-9.2-arm64-linux/
# /S32DS/build_tools/gcc_v10.2/gcc-10.2-arm64-linux/
# /S32DS/build_tools/gcc_v11.4/gcc-11.4-arm64-linux/
# /S32DS/tools/gdb_arm/arm64-linux/
```

Appendix B. Performance Considerations

The following suggestions will help you keep the S32 Design Studio tools running at a respectable performance level.

1. To maximize the performance, the S32 Design Studio tools should be installed on a computer with the recommended configuration. While the tools will operate on a computer with the operational minimum configuration, limited hardware will restrict its ability to function at desired performance levels.
2. Close unused projects. Eclipse caches files for all open projects in the workspace. If you need multiple projects open, try to limit the number of projects to no more than 10.

3. The Eclipse IDE provides several options that provide user assistance tools. These options, however, use memory and CPU resources. If the performance is slow, and you do not need these options, turn them off.
4. Scalability mode options configure how Eclipse deals with large source files and may affect the overall performance of the IDE. Turn off these options to maximize the performance. The following scalability mode options are available: editor live parsing, semantic highlighting, syntax coloring, parsing–based content assist proposals, and content assist auto activation.

   To disable:

   a. Choose **Window** > **Preferences**
   b. Expand **C/C++** > **Editor** > **Scalability**
   c. Uncheck **Enable all scalability mode options**
5. Content Assist Auto Activation can reduce the number of keystrokes one must type to create the code. The Content Assist plug-in consists of components that predict what keystroke you may type next, based on the current context, scope and prefix. Turn off this predictor to maximize the performance.

   To disable:

   a. Choose **Window** > **Preferences**
   b. Expand **C/C++** > **Editor** > **Content Assist**
   c. Uncheck all the options for **Auto-Activation**
6. For faster serial communication response (for example, UART connection), you can change the OS latency timer settings. The latency timer is a form of time-out mechanism for the read buffer of FTDI devices. The default value for the latency timer is 16 ms. You can decrease the latency value to maximize the performance:

   - On Windows, go to the port properties, open advanced port settings and select the required latency value from the drop-down menu.
   - On Linux, open the terminal and set the required value using the `echo` or `setserial` commands. For example:

     ```
     echo 2 > /sys/bus/usb-serial/devices/ttyUSB0/latency_timer
     ```

7. Problem Reporting Instructions

7.1. Problem Reporting Instructions

The S32 Design Studio for S32 Platform general issues are tracked through the S32DS Public NXP Community space: https://community.nxp.com/community/s32/s32ds.

For confidential cases and cases which cannot be publicly shared on NXP Community please follow the steps described here: https://community.nxp.com/docs/DOC-329745.

For cases with PEmicro submit a support request: https://www.pemicro.com/support/.

8. Known Issues and Workarounds

Appendix A. Known issues and Workarounds

- **Conditional watchpoints and breakpoints**: Conditional breakpoints and watchpoints, including those using ignore counts, may not work in some cases.

  Workaround: Avoid using conditions for breakpoints and watchpoints, instead check for condition in the code and set a normal breakpoint.
- **Disassembly view** content might be destroyed occasionally.

  Workaround: Close the Disassembly view and open it again by selecting **Window** > **Show View** > **Disassembly** on the menu.
- **Hyperlinks** may not work correctly when Microsoft Edge (early versions) is configured as the default browser.

  Workaround: Set a different browser as the default one.
- **Views** are not updated instantly after executing a command in the GDB console.

  Workaround: Use the `step` command to see refreshed views.
- The **Getting Started** page may be displayed blank in Linux.

  Workaround: Install Webkit1 for GTK2 using the following command: `sudo apt-get install libwebkitgtk-1.0-0`
- The **Getting Started** page may fail to load content at startup of S32 Design Studio for S32 Platform 3.6.0 RFP. “ The page can't be displayed ” error message may be shown instead.

  Workaround: Refresh the **Getting Started** page by pressing the F5 key.
- **Duplicated error dialog**: When adding more watchpoints than supported by the device, the popup box with the “ Not enough hardware resources for processing ” error message is displayed twice.

  Workaround: Close the popup box, then close it again.
- **Uninstallation of P&E drivers**: The P&E Device Drivers are not uninstalled with uninstallation of S32 Design Studio. After that, an attempt to uninstall them manually can cause errors.

  Workaround: If you do not need the P&E drivers for other products and devices, uninstall them before the product uninstallation.
- **Stepping over the try-catch block fails on a VDK**: When debugging a project on a VDK, an attempt to step over the try-catch block fails.

  Workaround: The issue is specific for projects compiled with NewLib. Recreate a project and select NewLib Nano as the library.
- **Watchpoints set on complex data types are not hit**: When debugging on a target connected with S32 Debug Probe, a debug session ignores a watchpoint set on a variable of a complex data type (such as a structure or other). A watchpoint set on an item of a basic data type inside a complex variable works correctly.

  Workaround: Avoid setting watchpoints at complex data types.

- **USB connection failure when debugging with S32 Debug Probe on a Linux VM**: A debug session fails on a Linux virtual machine with S32 Debug Probe connected through USB. Debugging with S32 Debug Probe connected through Ethernet can be done successfully.

  Workaround: Connect the probe to USB. Set up the debug configuration to use the Ethernet connection rather than USB. Specify the virtual IP address of the probe in the connection settings of the debug configuration. To obtain the required value, run the following command:

  ```
  sudo ifconfig -a
  ```

  The displayed output includes a section for the virtual Ethernet interface created on Linux for communication with the probe over the physical USB link. The *inet addr* value in this section is a local IP address starting with “ 169.254 ” . Take the *HWAddr* value in the section and convert the last two bytes into decimal notation. Append the resulting numbers to “ 169.254 ” to obtain the required virtual IP address.
- **Debugging on a VDK may fail**: Some errors may occur due to unexpected termination of a debug session or unsupported registers and watchpoints, etc.

  Workaround: Run a debug session in Virtualizer Studio.
- **Creating new project in the root of drive C:\ fails**: When creating a new project in the root directory, the project wizard shows the "Can't create directory: C:\project_name" error message. Windows does not allow user to create files in this location.

  Workaround: Create a subdirectory in the root directory (for example, C:\projects) and specify this location in the project creation wizard.
- **Register variable is not shown in memory view**: If the compiler locates the variable in register the debugger cannot show it in the memory view. "View memory" selection for this variable generates the error message: "Can't view memory on &(counter)".

  Workaround: Information which register is assigned for the variable can be found in:

  # the **Disassembly** view,
  # the error message in the **Expression** view. If "**&counter**" was added in the **Expression** view then the error message contains the information like this:

  ```
  Failed to execute MI command:
  -data-evaluate-expression *(&counter)
  Error message from debugger back end:
  Address requested for identifier "counter" which is in
   register $r<reg_num>
  ```
- **GHS project with SDK build failed after renaming of the project**: Header files are copied to the project, and rename process affects the paths. GHS build system believes that the files are not changed and does not update them during building - this is the GHS build system specific.

  Workaround: For GHS projects after renaming of the project activate clean and refresh procedures before building.

  For GHS projects with SDK after renaming of the project activate clean and refresh procedures before building.

GHS project without SDK build failed after renaming of the project: Header files are copied to the project, and rename process affects the paths. GHS build system believes that the files are not changed and does not update them during building - this is the GHS build system specific.

Workaround: For GHS projects after renaming of the project activate clean and refresh procedures before building.

For GHS projects without SDK after renaming of the project activate clean and refresh procedures before building.

- **Headless build system gives the "Managed Build system manifest file error"**: this error appears if the GHS toolchain is not installed. It is the Eclipse build system specific, but it is not an error technically and does not affect a build success.

  Workaround: To further avoid this error messages, install the GHS toolchain.
- **GHS project build gives the "Unresolved inclusion" warning when including a standard C library**: GHS toolchain specific implementation does not contain build-in spec detector.

  Workaround: Not no break C dialect specification, we recommend to add path to include files manually: Open project properties, go to **C/C++ General** > **Preprocessor Include Paths, Macros etc.** > **CDT User Setting Entries** and add an **Include Directory** with the path to GHS folder, e.g. `${S32DS_GHS_PATH}\ansi`, and the **Contains system headers** option selected.

- **S32 Debugger Program counter does not support HEX-value setting**.

  Workaround: When creating a **Debug Configuration**, at **Startup** tab use one of the following options:

  # use label or function name instead for the **Set program counter at** field in **Runtime Options**,
  # or uncheck the **Set program counter at** and add the following commands to **Run commands** field:

  ```
  set $pc=<HEX-value>
  -exec-until *<HEX-value>
  ```

- **Debugging on S32 Debugger may fail**: **Variables** view may pose a problem when entering some function with uninitialized pointers as it generates random access which may corrupt the debugger. In some cases the invalid access may be generated when debugging optimized code (GHS toolchain) where pointer location shared with some variable.

  Workaround: Close the **Variables** view or make it inactive for critical part of code (until pointers are not initialized in code).
- **Debugging on S32 Debugger**: **Variables** view and **Expressions** view may not display the value of some variables correctly if those variables have been used in macros.

  Workaround: Use the **Details** number format in these views as it is the only format that will display the correct value in this context.

- **Step out work incorrect with dbg_derived_types source**: The GHS toolchain does not generate DWARF Call Frame information that is suitable for unwinding the call stack. For third-party debuggers incapable of unwinding the call stack through an alternate means

(disassembling code), this may nullify the ability to walk the call stack or view variables saved in previous frames. As a workaround S32DS removes the debug frames info to force GDB client to recreate the frames. In some cases this method may not work and lead to incorrect behaviour for **Step out** command.

Workaround: Set a breakpoint at function end and run to breakpoint.

- **Project with huge amount of macros freezes S32DS IDE**: The huge count of macros is critical for indexing sources. Editing file with many macros causes 100% CPU usage and freezes S32DS IDE.

  Workaround: Manually tune `s32d.ini` file - remove the following lines:

  ```
  -Xms256m
  -Xmx2048m
  ```

- **Debugging of Cortex A53 projects with Diab Compiler**: Debug session stops with "No source available for "main() at". It is due to Diab version 7.0.3 have some limitations on debugging. Debug info for C files isn’t loaded in time. After stopping at the main function, several assembler commands must be executed before debug info will be loaded.

  Workaround: Perform some stepping within the debug session.
- **Projects renaming out of the workspace is not supported**: S32DS can't perform its custom rename procedure for a project out of the workspace used in the session. The eclipse default renaming will get the wrong result (problems with launch configurations and other related files).

  Workaround: Use one of the following options:

  # Copy the project to wokspace (use file system options).
  # Import the prioject with copying files to workspace.
  # Change to another workspace with the project to rename (when using several workspaces).
- S32DS has multiuser install issue on Linux.

  Workaround: To install the S32 Design Studio for multiple users perform the following steps:

  # Under the user with sudo rights:
    # Install the S32 Design Studio to the `/opt/NXP/S32DS.3.x` folder,
    # Change permission:

      ```
      sudo chmod -R 770 /opt/NXP/S32DS.3.x
      ```

    # Run the S32 Design Studio from the `/opt/NXP/S32DS.3.4` folder and close,
    # Change permission:

      ```
      sudo chmod 770 /dev/shm/cll_mutex
      sudo chmod -R 770 /opt/NXP/S32DS.3.x/eclipse/
      configuration
      ```

    # Add the second S32DS user without sudo rights to the first user's group:

      ```
      sudo usermod -aG <sudo_user_name> <S32DS_user_name>
      ```

Switch user to the <S32DS_user_name>,
Run the S32 Design Studio from the `/opt/NXP/S32DS.3.x` folder.

- **Connecting via telnet to the S32 Debug Probe configuration console fails**: When connecting multiple S32 Debug Probes simultaneously to a Linux host using USB, *telnet* will only be able to connect to the first device.

  Workaround: There are two options to access S32 Debug Probe configuration console:

  1. Connect using the serial port device file(s) located at: `/dev/ttyACM0` (first probe), `/dev/ttyACM1` (second probe), etc. with a program such as *screen*.
  2. Connect using telnet by adding temporary IP route table entries to the Linux host for the S32 Debug Probes:

     a. Identify the virtual IP address of the probes running the following command:

     ```
     sudo ifconfig -a
     ```

     The displayed output includes a section for the virtual Ethernet interface created on Linux for communication with the probe over the physical USB link. The *inet addr* value in this section is a local IP address starting with “169.254”. Take the *ether* value in the section and convert the last two bytes into decimal notation. Append the resulting numbers to “169.254” to obtain the required virtual IP address. Example: ether value *62:9F:B4:C6:D2:E3* translates to IP address *169.254.210.227*

     b. Identify the IP route matching each probe's local IP address using the IP route's *src* field. To list the existing IP routes use the following command:

     ```
     ip route
     ```

     Example of S32 Debug probe route:

     ```
     169.254.0.0/16 dev enx629fb4c6d2e3 proto kernel scope link
      src 169.254.186.146 metric 101
     ```

     c. Add a new route by replacing the *169.254.0.0/16* subnet in the existing route with the calculated virtual IP address for each probe using the following command:

     ```
     sudo ip route add <new_ip_route>
     ```

     Example:

     ```
     sudo ip route add 169.254.210.227 dev enx629fb4c6d2e3 proto
      kernel scope link src 169.254.186.146 metric 101
     ```

     d. Connect to S32 Debug probe using telnet

- Move to line in DIAB might not work accordingly

  Workaround: Don't change PC in projects compiled with DIAB.

- Launching S32Debug Group on Secure parts will generate NPE error

  Workaround: Fill Password in S32Debugger Configuration

- For M7 cores, during the trace collection some other packages might be generated outside of the actual execution of the core.
- Folders are not excluded if SDK is migrated on a S32DS 3.4.1 imported project.

  Workaround: An easy workaround for this issue would be to reload the SDK and exclude the old SDK folder from build path after the migration to a newer version of SDK. This would correctly exclude the backup folders.
- The scenario in which a user sets the environment variables PYTHONHOME and PYTHONPATH is not supported, as it can lead to unexpected results.

  Workaround: These two environment variables should not be set (user or system level) when working with S32 Design Studio.

Note: There are some issues which are introduced by Eclipse CDT and are therefore reproduced in the S32 Design Studio. These issues might be fixed when the fix is available in the future version of Eclipse CDT and the S32 Design Studio migrates to the updated CDT.

- The **Peripheral View** is crashing, when applying the selection on launch, in **Debug View**.
- When debug multicore launch and add some Memory Monitor addresses and terminate&relaunch the debug session, only addresses added on master core remain saved. If each core is terminated separately, the memory addresses are saved correctly.

  Workaround: Terminate the debug sessions on each core individually, not terminate all.
- **S32Trace shows no trace has been collected for application built with GCC 11.4:**

  # S32Trace does not support applications with DWARF-5 debug format, and will fail to open the application file when decoding. Starting with GCC 11.4 the default format is DWARF-5, and even if we change this for our project back to DWARF-4, the other precompiled code in the application will still be in DWARF-5 and block the decoding.
- Command line project creation and compilation with RTD/SDK option might require additional setup.
- **Import C/C++ Executable as S32DS project with Lauterbach:** When importing C/C++ Executable with Lauterbach, the Configuration File and PRACTICE script fields in the launch configuration are empty.

  Workaround: **To debug the imported ELF project** , manually fill in the required absolute paths for the following fields in the launch configuration: Configuration File, Initial Working Directory, and PRACTICE Script.
- **Import C/C++ Executable as S32DS project with Tasking:** When importing C/C++ Executable with Tasking, the Workspace field in the launch configuration is empty.

  Workaround: **To debug the imported ELF project** , manually fill in the required absolute path for the Workspace field in the launch configuration.
- **Cannot install Eclipse p2 installation units on Ubuntu 20.04 distribution when "Always trust all authorities" or "Always trust all content" is selected**. For these cases, "Always Trust Everything Confirmation" window becomes hidden and user cannot access it. This is an Eclipse issue documented here: https://github.com/eclipse-equinox/p2/issues/570.

  Workaround: Use "Select All" option in all windows instead of "Always trust all authorities" or "Always trust all content".

- **S32 Debugger** and/or **S32 Trace** no longer work after update fails due to "access denied" error:

  When installing the Platform package in S32DS Extensions and Updates after having used S32 Debugger or S32 Trace, an error message may appear stating that access is denied to files in use. After this, S32 Debugger and S32 Trace may no longer function.

  Workaround: Restart S32 Design Studio and install the update again, as instructed in the dialog. Once the update completes successfully, functionality should be restored to normal.

9. Revision history

Revision Table

Table 1: Revision history

Revision Number	Changes	Date
1.0	New product launch	22.10.2024

How to Reach Us:

Home Page:

nxp.com

Web Support:

nxp.com/support

Information in this document is provided solely to enable system and software implementers to use NXP products. There are no express or implied copyright licenses granted hereunder to design or fabricate any integrate circuits based on the information in this document. NXP reserves the right to make changes without further notice to any products herein.

NXP makes no warranty, representation, or guarantee regarding the suitability of its products for any particular purpose, nor does NXP assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation consequential or incidental damages. “Typical” parameters that may be provided in NXP data sheets and/or specifications can and do vary in different applications, and actual performance may vary over time. All operating parameters, including “typicals”, must be validated for each customer application by customer's technical experts. NXP does not convey any license under its patent rights nor the rights of others. NXP sells products pursuant to standard terms and conditions of sale, which can be found at the following address: nxp.com/SalesTermsandConditions.

While NXP Semiconductors has implemented advanced security features, all products may be subject to unidentified vulnerabilities. Customers are responsible for the design and operation of their applications and products to reduce the effect of these vulnerabilities on customer’s applications and products, and NXP Semiconductors accepts no liability for any vulnerability that is discovered. Customers should implement appropriate design and operating safeguards to minimize the risks associated with their applications and products.

For more information, please visit: http://www.nxp.com

For sales office addresses, please send an email to: salesaddresses@nxp.com

Revision: 1.0, 11/2024

Document number: S32DS360